МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ПОВОЛЖСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра ИиСП

Отчет
по лабораторной работе № 7

по дисциплине «Машинно-зависимые языки программирования»
Вариант 4

Выполнил: ст. гр. ПС-
Петялин А.Н.
Проверил: доцент, доцент кафедры ИиСП Баев А.А.

г. Йошкар-Ола
2025

Цель работы: изучить работу АЦП и блока UART.

Задания на лабораторную работу: написать программы с использованием аналогово-цифровых преобразований и интерфейса асинхронной последовательной передачи данных.

1. Теоретические сведения

Контроллер ATmega328 имеет в своем составе модуль АЦП. Этот модуль позволяет контроллеру обрабатывать аналоговые вели чины, что существенно расширяет сферу его применения. Согласно техническому описанию контроллера, АЦП имеет разрядность 10 бит с абсолютной погрешностью ±2 LSB (Least Significant Bit – младший значащий бит), то есть теоретически напряжение может быть измерено с точностью около 0,2 %. Но на практике точность производимых измерений обычно значительно ниже, так как на неё влияет множество факторов, таких как стабильность питания и опорного напряжения, нагрев контроллера, помехи, проникновение цифровых помех в аналоговую часть и так далее.

Отличительные особенности данного АЦП:

- 10-разрядное разрешение;
- интегральная нелинейность 0,5 мл. разр.;
- абсолютная погрешность ±2 мл. разр.;
- время преобразования 13 - 260 мкс;
- 8 (6) мультиплексированных однополярных входов (количество зависит от корпуса микросхемы);
- представление результата с левосторонним или правосторонним выравниванием в 16-разрядном слове;
- диапазон входного напряжения АЦП 0…VCC;
- выборочный внутренний ИОН на 1,1 В;
- режимы одиночного преобразования и автоматического пере запуска;
- прерывание по завершении преобразования АЦП.

Еще одним важным блоком контроллера ATmega328 является интерфейс UART. Это интерфейс асинхронной последовательной передачи данных. В отличие от синхронного интерфейса SPI, в интерфейсе UART не используется линия передачи тактовых сигналов. Вместо этого приемник и передатчик должны быть синхронизированы по частоте так, чтобы прием и передача

велись с одной, заранее оговоренной скоростью. Скорость UART для конкретного соединения указывается в бодах. Существует общепринятый ряд стандартных скоростей: 300; 600; 1200; 2400; 4800; 9600; 19200; 38400; 57600; 115200; 230400; 460800; 921600 бод. Также UART позволяет связать между собой только два устройства.

2. Практическая часть

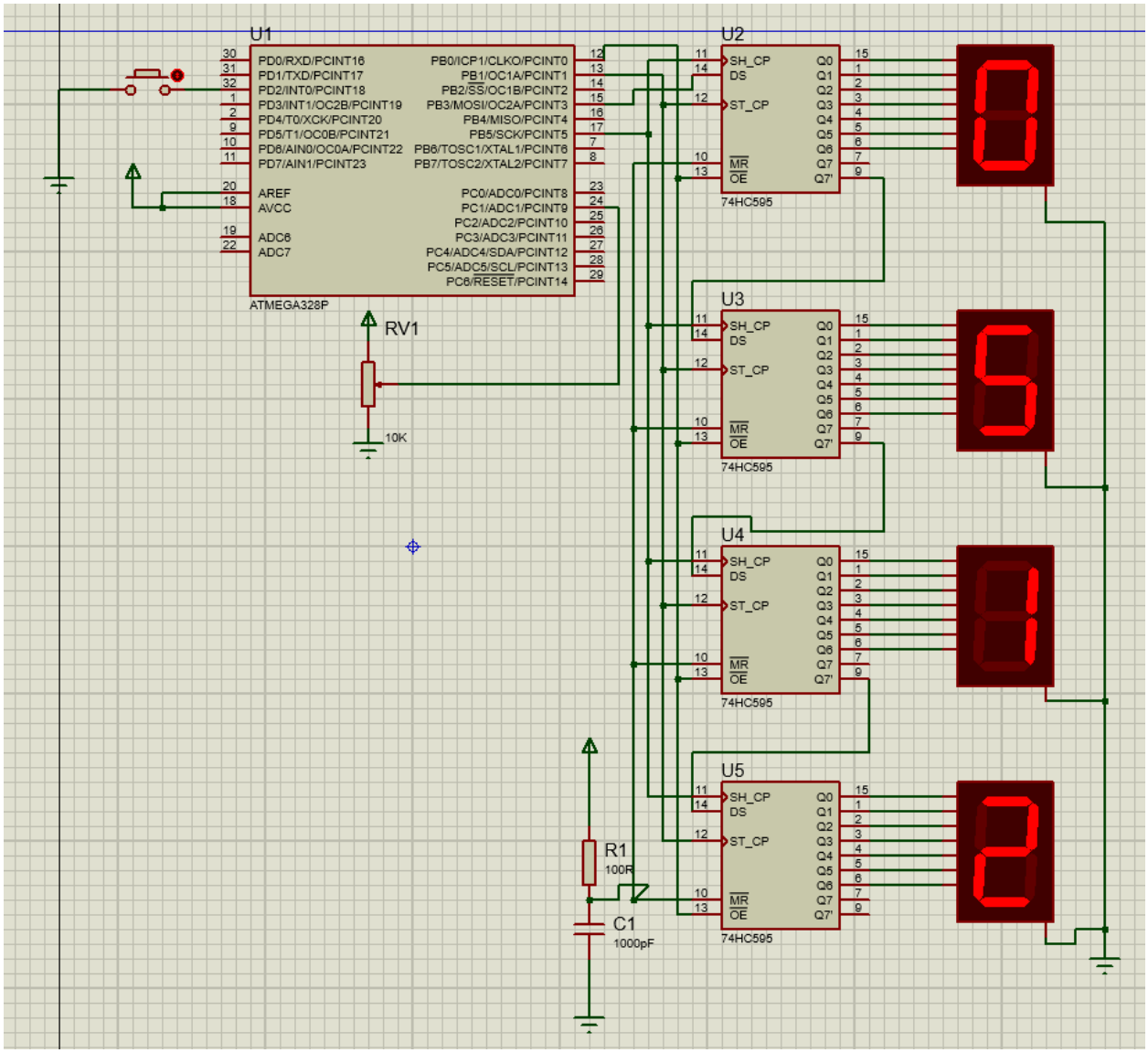
Код первой программы:

```
#define F_CPU 1000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
uint8_t segments[] = {
	// GFEDCBA
	0b00111111, // 0 - A, B, C, D, E, F
	0b00000110, // 1 - B, C
	0b01011011, // 2 - A, B, D, E, G
	0b01001111, // 3 - A, B, C, D, G
	0b01100110, // 4 - B, C, F, G
	0b01101101, // 5 - A, C, D, F, G
	0b01111101, // 6 - A, C, D, E, F, G
	0b00000111, // 7 - A, B, C
	0b01111111, // 8 - A, B, C, D, E, F, G
	0b01101111, // 9 - A, B, C, D, F, G
};
void InitPorts(void);
void InitTimer1(void);
void Bin2Dec(uint16_t data);
void SendData(uint8_t data);
void DisplayData(uint16_t data);
void InitSPI(void);
void InitADC(void);
volatile uint8_t bcd_buffer[] = {0,0,0,0};
volatile uint16_t display_val = 0;
int main(void)
{
	InitPorts();
	InitSPI();
	InitTimer1();
	EIMSK |= (1 << INT0);  //Enable INT0
	EICRA |= (1 << ISC01); //Trigger on falling edge of INT0
	InitADC();
	sei();
	//global interrupt enable
	PORTB &= ~(1 << PINB0); //OE = low (active)
	DisplayData(0);
	while(1)
```

```
        {
                DisplayData(display_val);
        }
}
//-------------------------------------------
ISR(TIMER1_COMPB_vect){
}
ISR(INT0_vect){
}
ISR(ADC_vect){
        display_val = ADC;
}
//-------------------------------------------
void InitPorts(void){
        DDRB = (1 << PINB0 | 1 << PINB1 | 1 << PINB3 | 1 << PINB5);
        DDRD = (0 << PIND2);
        PORTD |= (1 << PIND2);
}
void InitTimer1( void){
        TCCR1A = 0;
        //CTC mode -Clear Timer on Compare
        //prescaler = sys_clk/64
        TCCR1B = (1 << CS11 | 1 << CS10 | 1 << WGM12);
        TCNT1 = 0;
        //start value of counter
        TIMSK1 |= (1<<OCIE1B);
        OCR1A = 1562;
        OCR1B = 1562;
}
void Bin2Dec(uint16_t data){
        bcd_buffer[3] = (uint8_t)(data/1000);
        data = data % 1000;
        bcd_buffer[2] = (uint8_t)(data/100);
        data = data % 100;
        bcd_buffer[1] = (uint8_t)(data/10);
        data = data % 10;
        bcd_buffer[0] = (uint8_t)(data);
}
void SendData (uint8_t data){
        SPDR = data;
        while(!(SPSR & (1 << SPIF)));
}
void DisplayData (uint16_t data){
        Bin2Dec(data);
        PORTB &= ~(1 << PINB1);
```

```
		//clk_out = 0
		SendData(segments[bcd_buffer[0]]);
		SendData(segments[bcd_buffer[1]]);
		SendData(segments[bcd_buffer[2]]);
		SendData(segments[bcd_buffer[3]]);
		PORTB |= (1 << PINB1);
		//clk_out = 1
	}
	void InitSPI( void){
		DDRB |= (1 << PINB3 | 1 << PINB5);//configure MOSI and CLK as
out
		SPSR |= (1 << SPI2X); //Fclk = Fosc/2
		//SPI enable, master mode, MSB first, CPOL=0, CPHA=0
		SPCR = (1 << SPE | 1 << MSTR);
		//init values - DAT low, CLK low
		PORTB &= ~(1 << PINB3 | 1 << PINB5);
	}
	void InitADC( void){
		ADMUX = (1 << MUX0);
		//Align left, ADC1
		ADCSRB = (1 << ADTS2 | 1 << ADTS0); //Start on Timer1 COMPB
		//Enable, auto update, IRQ enable
		ADCSRA = (1 << ADEN | 1 << ADATE | 1 << ADIE);
	}
```

Схема в Proteus



Код второй программы:

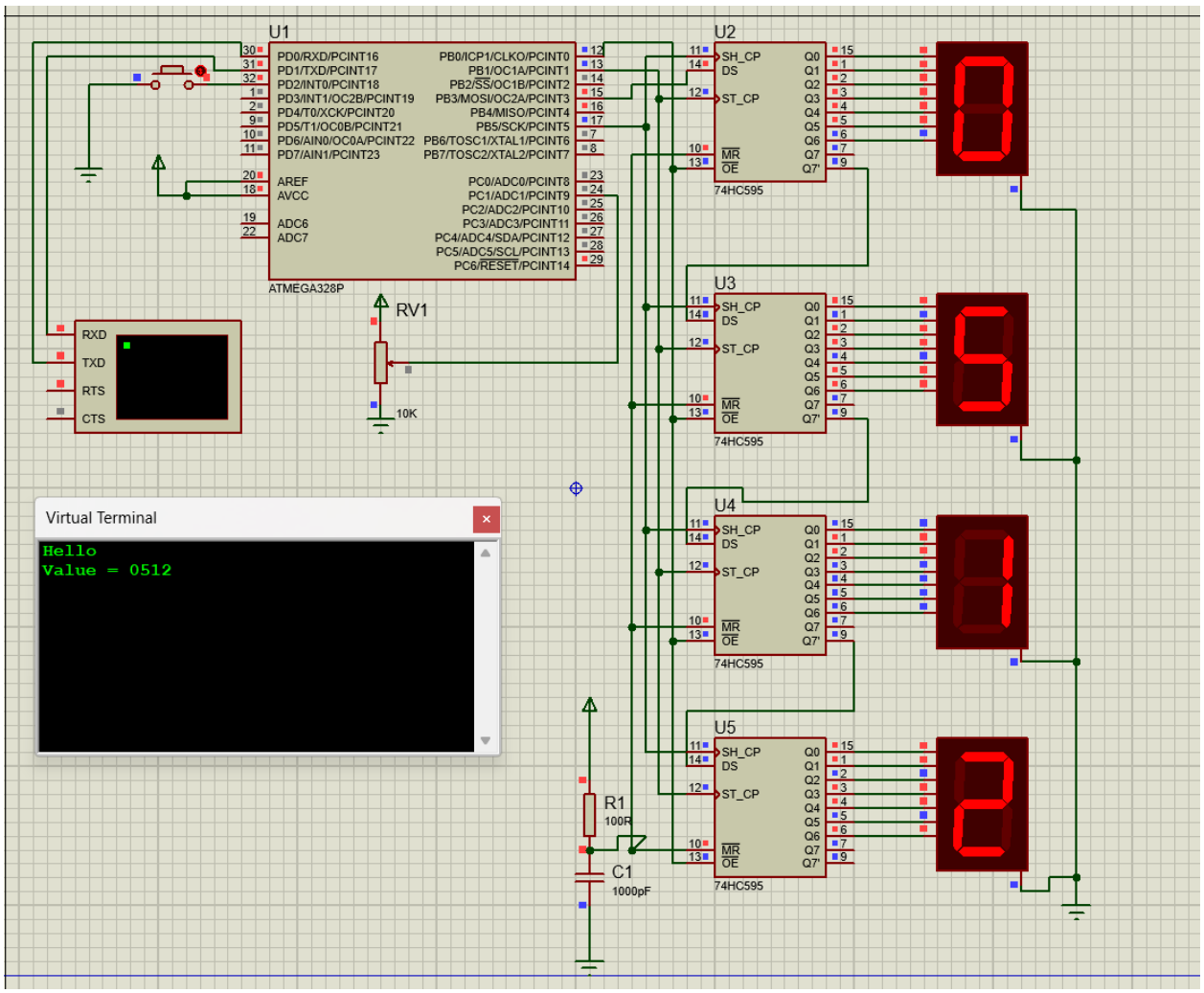
```
#define F_CPU 1000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
uint8_t segments[] = {
	// GFEDCBA
	0b00111111, // 0 - A, B, C, D, E, F
	0b00000110, // 1 - B, C
	0b01011011, // 2 - A, B, D, E, G
	0b01001111, // 3 - A, B, C, D, G
	0b01100110, // 4 - B, C, F, G
	0b01101101, // 5 - A, C, D, F, G
	0b01111101, // 6 - A, C, D, E, F, G
	0b00000111, // 7 - A, B, C
	0b01111111, // 8 - A, B, C, D, E, F, G
	0b01101111, // 9 - A, B, C, D, F, G
};
void InitPorts(void);
void InitTimer1(void);
void Bin2Dec(uint16_t data);
void SendData(uint8_t data);
void DisplayData(uint16_t data);
void InitSPI(void);
void InitADC(void);
void InitUSART(void);
void SendString(char* buffer);
void SendChar(char symbol);
volatile uint8_t bcd_buffer[] = {0,0,0,0};
volatile uint16_t display_val = 0;
int main(void)
{
	InitPorts();
	InitSPI();
	InitTimer1();
	EIMSK |= (1 << INT0);  //Enable INT0
	EICRA |= (1 << ISC01); //Trigger on falling edge of INT0
	InitADC();
	InitUSART();
	sei();
	//global interrupt enable
	PORTB &= ~(1 << PINB0); //OE = low (active)
	DisplayData(0);
	SendString("Hello\r\n");
```

```
        while(1)
        {
                DisplayData(display_val);
        }
}
//-------------------------------------------
ISR(TIMER1_COMPB_vect){
}
ISR(INT0_vect){
        SendString("Value = ");
        SendChar(0x30 + bcd_buffer[3]);
        SendChar(0x30 + bcd_buffer[2]);
        SendChar(0x30 + bcd_buffer[1]);
        SendChar(0x30 + bcd_buffer[0]);
        SendString("\r\n");
}
ISR(USART_RX_vect){
        if(UDR0 == 0x20)
        {
                SendString("It was SPACE button\r\n");
        }
}
ISR(ADC_vect){
        display_val = ADC;
}
//-------------------------------------------
void InitPorts(void){
        DDRB = (1 << PINB0 | 1 << PINB1 | 1 << PINB3 | 1 << PINB5);
        DDRD = (0 << PIND2);
        PORTD |= (1 << PIND2);
}
void InitTimer1( void){
        TCCR1A = 0;
        //CTC mode -Clear Timer on Compare
        //prescaler = sys_clk/64
        TCCR1B = (1 << CS11 | 1 << CS10 | 1 << WGM12);
        TCNT1 = 0;
        //start value of counter
        TIMSK1 |= (1<<OCIE1B);
        OCR1A = 1562;
        OCR1B = 1562;
}
void Bin2Dec(uint16_t data){
        bcd_buffer[3] = (uint8_t)(data/1000);
        data = data % 1000;
```

```
		bcd_buffer[2] = (uint8_t)(data/100);
		data = data % 100;
		bcd_buffer[1] = (uint8_t)(data/10);
		data = data % 10;
		bcd_buffer[0] = (uint8_t)(data);
	}
	void SendData (uint8_t data){
		SPDR = data;
		while(!(SPSR & (1 << SPIF)));
	}
	void DisplayData (uint16_t data){
		Bin2Dec(data);
		PORTB &= ~(1 << PINB1);
		//clk_out = 0
		SendData(segments[bcd_buffer[0]]);
		SendData(segments[bcd_buffer[1]]);
		SendData(segments[bcd_buffer[2]]);
		SendData(segments[bcd_buffer[3]]);
		PORTB |= (1 << PINB1);
		//clk_out = 1
	}
	void InitSPI( void){
		DDRB |= (1 << PINB3 | 1 << PINB5);//configure MOSI and CLK as
out
		SPSR |= (1 << SPI2X); //Fclk = Fosc/2
		//SPI enable, master mode, MSB first, CPOL=0, CPHA=0
		SPCR = (1 << SPE | 1 << MSTR);
		//init values - DAT low, CLK low
		PORTB &= ~(1 << PINB3 | 1 << PINB5);
	}
	void InitADC( void){
		ADMUX = (1 << MUX0);
		//Align left, ADC1
		ADCSRB = (1 << ADTS2 | 1 << ADTS0); //Start on Timer1 COMPB
		//Enable, auto update, IRQ enable
		ADCSRA = (1 << ADEN | 1 << ADATE | 1 << ADIE);
	}
	void InitUSART(){
		UCSR0B = (1 << RXEN0 | 1 << TXEN0);
		UCSR0C = (1 << UCSZ01 | 1 << UCSZ00);
		UBRR0H = 0;
		UBRR0L = 0x0C;
	}
	void SendChar(char symbol){
		while(!(UCSR0A & (1<<UDRE0))); //wait until flag cleared
```

```
        UDR0 = symbol;
}
void SendString(char* buffer){
        while(*buffer != 0){
                SendChar(*buffer++);
        }
}
```

Схема в Proteus



Оптимизация кодов

Первый код:

```
#define F_CPU 1000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
```

```
uint8_t segments[] = {
	// GFEDCBA
	0b00111111, // 0 - A, B, C, D, E, F
	0b00000110, // 1 - B, C
	0b01011011, // 2 - A, B, D, E, G
	0b01001111, // 3 - A, B, C, D, G
	0b01100110, // 4 - B, C, F, G
	0b01101101, // 5 - A, C, D, F, G
	0b01111101, // 6 - A, C, D, E, F, G
	0b00000111, // 7 - A, B, C
	0b01111111, // 8 - A, B, C, D, E, F, G
	0b01101111, // 9 - A, B, C, D, F, G
};
void InitPorts(void);
void InitTimer1(void);
void Bin2Dec(uint16_t data);
void SendData(uint8_t data);
void DisplayData(uint16_t data);
void InitSPI(void);
void InitADC(void);
volatile uint8_t bcd_buffer[] = {0,0,0,0};
volatile uint16_t display_val = 0;
volatile uint8_t flag = 0;
int main(void)
{
	InitPorts();
	InitSPI();
	InitTimer1();
	EIMSK |= (1 << INT0);  //Enable INT0
	EICRA |= (1 << ISC01); //Trigger on falling edge of INT0
	InitADC();
	sei();
	//global interrupt enable
	PORTB &= ~(1 << PINB0); //OE = low (active)
	DisplayData(0);
	while(1)
	{
		if (flag == 1)
		{
			DisplayData(display_val);
			flag = 0;
		}

	}
}
```

```
//-------------------------------------------
ISR(TIMER1_COMPB_vect){
}
ISR(ADC_vect){
	display_val = ADC;
	flag = 1;
}
//-------------------------------------------
void InitPorts(void){
	DDRB = (1 << PINB0 | 1 << PINB1 | 1 << PINB3 | 1 << PINB5);
	DDRD = (0 << PIND2);
	PORTD |= (1 << PIND2);
}
void InitTimer1( void){
	TCCR1A = 0;
	//CTC mode -Clear Timer on Compare
	//prescaler = sys_clk/64
	TCCR1B = (1 << CS11 | 1 << CS10 | 1 << WGM12);
	TCNT1 = 0;
	//start value of counter
	TIMSK1 |= (1<<OCIE1B);
	OCR1A = 1562;
	OCR1B = 1562;
}
void Bin2Dec(uint16_t data){
	bcd_buffer[3] = (uint8_t)(data/1000);
	data = data % 1000;
	bcd_buffer[2] = (uint8_t)(data/100);
	data = data % 100;
	bcd_buffer[1] = (uint8_t)(data/10);
	data = data % 10;
	bcd_buffer[0] = (uint8_t)(data);
}
void SendData (uint8_t data){
	SPDR = data;
	while(!(SPSR & (1 << SPIF)));
}
void DisplayData (uint16_t data){
	Bin2Dec(data);
	PORTB &= ~(1 << PINB1);
	//clk_out = 0
	SendData(segments[bcd_buffer[0]]);
	SendData(segments[bcd_buffer[1]]);
	SendData(segments[bcd_buffer[2]]);
	SendData(segments[bcd_buffer[3]]);
```

```
		PORTB |= (1 << PINB1);
		//clk_out = 1
	}
	void InitSPI( void){
		DDRB |= (1 << PINB3 | 1 << PINB5);//configure MOSI and CLK as
out
		SPSR |= (1 << SPI2X); //Fclk = Fosc/2
		//SPI enable, master mode, MSB first, CPOL=0, CPHA=0
		SPCR = (1 << SPE | 1 << MSTR);
		//init values - DAT low, CLK low
		PORTB &= ~(1 << PINB3 | 1 << PINB5);
	}
	void InitADC( void){
		ADMUX = (1 << MUX0);
		//Align left, ADC1
		ADCSRB = (1 << ADTS2 | 1 << ADTS0); //Start on Timer1 COMPB
		//Enable, auto update, IRQ enable
		ADCSRA = (1 << ADEN | 1 << ADATE | 1 << ADIE);
	}
```

Здесь мы убираем вектор прерываний по INT0, так как кнопка, подключенная к нужному пину, не используется. Добавляем флаг, чтобы отправка данных происходила только при их изменении.

Второй код:

```
#define F_CPU 1000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
uint8_t segments[] = {
	// GFEDCBA
	0b00111111, // 0 - A, B, C, D, E, F
	0b00000110, // 1 - B, C
	0b01011011, // 2 - A, B, D, E, G
	0b01001111, // 3 - A, B, C, D, G
	0b01100110, // 4 - B, C, F, G
	0b01101101, // 5 - A, C, D, F, G
	0b01111101, // 6 - A, C, D, E, F, G
	0b00000111, // 7 - A, B, C
	0b01111111, // 8 - A, B, C, D, E, F, G
	0b01101111, // 9 - A, B, C, D, F, G
};
```

```
void InitPorts(void);
void InitTimer1(void);
void Bin2Dec(uint16_t data);
void SendData(uint8_t data);
void DisplayData(uint16_t data);
void InitSPI(void);
void InitADC(void);
void InitUSART(void);
void SendString(char* buffer);
void SendChar(char symbol);
volatile uint8_t bcd_buffer[] = {0,0,0,0};
volatile uint16_t display_val = 0;
volatile uint8_t button_flag = 0;
volatile uint8_t data_flag = 0;
int main(void)
{
	InitPorts();
	InitSPI();
	InitTimer1();
	EIMSK |= (1 << INT0);  //Enable INT0
	EICRA |= (1 << ISC01); //Trigger on falling edge of INT0
	InitADC();
	InitUSART();
	sei();
	//global interrupt enable
	PORTB &= ~(1 << PINB0); //OE = low (active)
	DisplayData(0);
	SendString("Hello\r\n");
	while(1)
	{
		if (button_flag == 1)
		{
			SendString("Value = ");
			SendChar(0x30 + bcd_buffer[3]);
			SendChar(0x30 + bcd_buffer[2]);
			SendChar(0x30 + bcd_buffer[1]);
			SendChar(0x30 + bcd_buffer[0]);
			SendString("\r\n");
			button_flag = 0;
		}
		if (data_flag == 1)
		{
			DisplayData(display_val);
			data_flag = 0;
		}
```

```
        }
}
//-------------------------------------------
ISR(TIMER1_COMPB_vect){
}
ISR(INT0_vect){
        button_flag = 1;
}
ISR(USART_RX_vect){
        if(UDR0 == 0x20)
        {
                SendString("It was SPACE button\r\n");
        }
}
ISR(ADC_vect){
        display_val = ADC;
        data_flag = 1;
}
//-------------------------------------------
void InitPorts(void){
        DDRB = (1 << PINB0 | 1 << PINB1 | 1 << PINB3 | 1 << PINB5);
        DDRD = (0 << PIND2);
        PORTD |= (1 << PIND2);
}
void InitTimer1( void){
        TCCR1A = 0;
        //CTC mode -Clear Timer on Compare
        //prescaler = sys_clk/64
        TCCR1B = (1 << CS11 | 1 << CS10 | 1 << WGM12);
        TCNT1 = 0;
        //start value of counter
        TIMSK1 |= (1<<OCIE1B);
        OCR1A = 1562;
        OCR1B = 1562;
}
void Bin2Dec(uint16_t data){
        bcd_buffer[3] = (uint8_t)(data/1000);
        data = data % 1000;
        bcd_buffer[2] = (uint8_t)(data/100);
        data = data % 100;
        bcd_buffer[1] = (uint8_t)(data/10);
        data = data % 10;
        bcd_buffer[0] = (uint8_t)(data);
}
void SendData (uint8_t data){
```

```
        SPDR = data;
        while(!(SPSR & (1 << SPIF)));
}
void DisplayData (uint16_t data){
        Bin2Dec(data);
        PORTB &= ~(1 << PINB1);
        //clk_out = 0
        SendData(segments[bcd_buffer[0]]);
        SendData(segments[bcd_buffer[1]]);
        SendData(segments[bcd_buffer[2]]);
        SendData(segments[bcd_buffer[3]]);
        PORTB |= (1 << PINB1);
        //clk_out = 1
}
void InitSPI( void){
        DDRB |= (1 << PINB3 | 1 << PINB5);//configure MOSI and CLK as out
        SPSR |= (1 << SPI2X); //Fclk = Fosc/2
        //SPI enable, master mode, MSB first, CPOL=0, CPHA=0
        SPCR = (1 << SPE | 1 << MSTR);
        //init values - DAT low, CLK low
        PORTB &= ~(1 << PINB3 | 1 << PINB5);
}
void InitADC( void){
        ADMUX = (1 << MUX0);
        //Align left, ADC1
        ADCSRB = (1 << ADTS2 | 1 << ADTS0); //Start on Timer1 COMPB
        //Enable, auto update, IRQ enable
        ADCSRA = (1 << ADEN | 1 << ADATE | 1 << ADIE);
}
void InitUSART(){
        UCSR0B = (1 << RXEN0 | 1 << TXEN0);
        UCSR0C = (1 << UCSZ01 | 1 << UCSZ00);
        UBRR0H = 0;
        UBRR0L = 0x0C;
}
void SendChar(char symbol){
        while(!(UCSR0A & (1<<UDRE0))); //wait until flag cleared
        UDR0 = symbol;
}
void SendString(char* buffer){
        while(*buffer != 0)
        {
                SendChar(*buffer++);
        }
}
```

Здесь мы полностью убираем работу программы из прерываний, для этого создаем две переменные: одну для отображения состояния кнопки(button_flag), другую для вывода данных при их изменении (data_flag).

Ответы на вопросы:

1. Что такое АЦП? Для чего он используется?

Ответ: АЦП(аналогово-цифровой преобразователь) – это модуль, позволяющий контроллеру обрабатывать аналоговые величины. Он используется для оцифровки сигналов.

2. Какие факторы влияют на точность работы АЦП?

Ответ: На точность работы АЦП влияют такие факторы, как стабильность питания и опорного напряжения, нагрев контроллера, помехи, проникновение цифровых помех в аналоговую часть.

3. Назовите виды погрешностей, возникающих при работе АЦП.

Ответ: При работе АЦП возникают такие погрешности, как:

1) Погрешность квантования, возникающая из-за преобразования входного напряжение в конечное число кодов. Погрешность квантования – интервал входного напряжения протяженностью 1 младший разряд (шаг квантования по напряжению), который характеризуется одним и тем же кодом. Всегда равен +-0,5 младшего разряда.

2) Абсолютная погрешность, максимальное отклонение реальной (без подстройки) функции преобрахования от реальной при любом коде. Является результатом нескольких эффектов: смещение, погрешность усиления, дифференциальная погрешность, нелинейность и погрешность квантования. Идеальное значение +- 0,5 младшего разряда, но зачастую +- 2 младших разряда.

3) Смещение, отклонение первого перехода (с 0x000 на 0x001) по сравнению с идеальным переходом (т.е. при 0,5 младшего разряда). Идеальное значение: 0 младшего разряда.

4) Погрешность усиления, после корректировки смещения погрешность усиления представляет собой отклонение последнего перехода (с 0x3FE на 0x3FF) от идеального перехода (т.е. отклонение при максимальном значении минус 1,5 младшего разряда). Идеальное значение: 0 младшего разряда.

5) Интегральная нелинейность, после корректировки смещения и погрешности усиления, представляет собой максимальное отклонение реальной функции преобразования от идеальной для любого кода. Идеальное значение : 0 младшего разряда.

6) Дифференциальная нелинейность, максимальное отклонение между шириной фактического кода (интервал между двумя смежными переходами) от ширины идеального кода (1 младшего разряда). Идеальной значение: 0 младшего разряда.

4. У АЦП ATmega168 есть функция, позволяющая выравнивать результат преобразования АЦП по левой границе. Для чего?

Ответ: Выравнивание слева полезно, когда нам достаточно восьмиразрядное разрешение, т.к. в этом случае необходимо считать только регистр ADCH.

5. Назовите возможные режимы работы АЦП.

Ответ: Есть два режима работы: одиночного преобразования и автоматического преобразования.

6. Что такое UART? Чем он отличается от SPI?

Ответ: UART – интерфейс асинхронной последовательной передачи данных. Он отличается от SPI тем, что в нем не использует линия передачи тактовых сигналов, вместо чего приемник и передатчик должны быть синхронизированы по частоте.UART позволяет связать между собой лишь 2 устройства.

7. Опишите процесс передачи байта через UART.

Ответ:Байт обрамляется стартовым и стоповым битом, служащие для синхронизации. Стартовый бит всегда 0, поэтому приемник UART ждёт перепада из 1 в 0 и отсчитывает от него временной промежуток в половину длительности бита (середина передачи стартового бита). Если в этот момент на входе всё ещё 0, то запускается процесс приёма минимальной посылки. Для этого приёмник отсчитывает 9 битовых длительностей подряд (для 8-бит данных) и в каждый момент фиксирует состояние входа. Первые 8 значений являются принятыми данными, последнее значение проверочное (стоп-бит). Значение стоп-бита всегда 1; если реально принятое значение иное, UART фиксирует ошибку.

8. Почему необходимо точно установить частоту работы UART?

Ответ: Для формирования временных интервалов передающий и приёмный UART имеют точные источники тактирования. Точность этих источников должна быть такой, чтобы сумма погрешностей (приёмника и передатчика) установки временного интервала от начала стартового импульса до середины стопового импульса не превышала половины (а лучше хотя бы четверти) битового интервала. Рекомендуемый допуск на точность тактирования UART составляет не более 1,5 %.

9. Каким образом можно использовать UART для подключения микроконтроллера к компьютеру?

Ответ: Интерфейс RS-232 может быть использован для подключения контроллера к персональному компьютеру. Однако в современных условиях интерфейс RS-232 повсеместно вытесняется интерфейсом USB, в то время как реализация USB присутствует лишь на более мощных контроллерах, а работа с ним достаточно трудоемка. Для разрешения данной проблемы выпускаются микросхемы-переходники с интерфейса USB на UART. Примером такой микросхемы может служить FT232RL. Данная микросхема позволяет подключить необходимое устройство к ПК через интерфейс UART путем создания на ПК виртуального COM-порта, работа с которым не отличается от работы с физическим COM-портом. Таким образом, с точки зрения целевого устройства, работа ведется через интерфейс UART.

10. Для чего при передаче данных с АЦП в данной работе к каждому полученному числу прибавлялось число 0x30?

Ответ: К данным прибавляется значение 0x30, потому что символы, выводимые в терминале, соответствуют кодировке ASCII, а в таблице ASCII символы цифр начинаются с позиции 0x30.

Выводы

Я научился пользоваться интерфейсом UART и изучил принцип работы АЦП.